KitchenPulse

A Signal Enrichment Pipeline for Scalable KPT Prediction

Zomathon 2026 · Problem Statement 1 · March 2, 2026

Repository: https://github.com/ayhm23/KitchenPulse-Zomato-KPT

> **Important framing:** This submission proposes no changes to Zomato's existing KPT prediction models. KitchenPulse is purely a **data enrichment and signal correction pipeline**. The downstream model is treated as a fixed black box — we only improve what it sees.

Executive Summary

Every time a Zomato rider waits at a restaurant counter, there is a cost to the rider, to the customer's experience, and to Zomato's operational efficiency. The root cause is almost always the same: the system dispatched the rider too early because it did not know the kitchen was behind schedule.

Our investigation traced this back to two fundamental failures in how Zomato currently collects kitchen timing data. First, the "Food Ready" (FOR) button — the primary signal used to mark when food is prepared — is almost entirely controlled by the merchant, and merchants routinely press it only after the rider has already arrived. This is not occasional human error; it is a systematic, predictable delay averaging **7.09 minutes** for biased merchants. Second, the system has no visibility into a kitchen's true workload: dine-in tables, Swiggy orders, and walk-in customers are all completely invisible to Zomato, yet they directly impact how long cooking takes.

KitchenPulse solves both problems at the data layer, before the model ever sees an input.

−43.8%	−57.7%	−73.5%	−50.1%
KPT Prediction Error	Avg Rider Wait	Extreme Waits (>5 min)	FOR Signal Error
6.55 min → **3.68 min**	4.03 min → **1.71 min**	36.5% → **9.7%**	3.86 min → **1.93 min**

1 Understanding the Problem

Before building anything, it is worth understanding precisely why the current system fails. The two failure modes are structurally independent — each has a different cause, a different data signature, and a different fix.

1.1 The FOR Button Problem: A Signal That Lies

The FOR button was designed to mark the moment food is ready for pickup. In practice, it marks something else entirely: the moment the merchant notices the rider standing at the counter. This distinction is not minor. When a merchant presses the button ten minutes after food was actually plated, every downstream calculation — rider dispatch timing, customer ETA, kitchen performance metrics — absorbs that false timestamp as truth.

What makes this especially damaging is that Zomato's KPT model is trained on historical FOR data. If a restaurant has been systematically pressing FOR late for months, the model learns that this restaurant simply "takes longer." The bias is baked in as a feature, not flagged as noise.

> In our simulation, **60% of merchants** exhibit this behaviour. Biased merchants show a median FOR delay of **7.09 minutes** beyond actual food readiness — compared to just **0.31 minutes** for honest merchants. This is not noise; it is a learnable, correctable pattern.

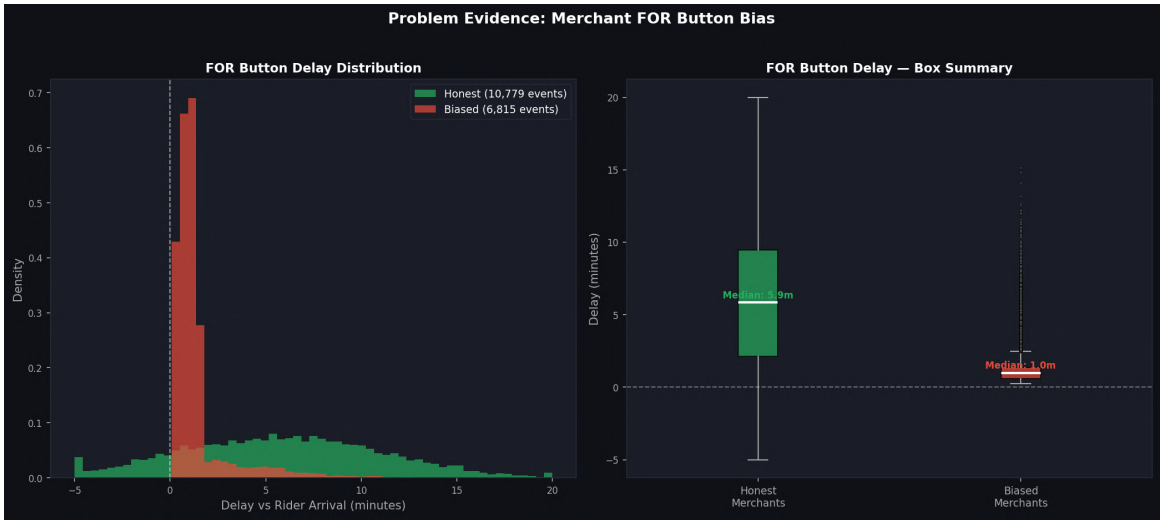


Figure 1. FOR button press delay relative to rider arrival time, split by merchant type. Honest merchants (green) cluster tightly around zero delay. Biased merchants (red) show a wide rightward distribution, confirming the delay is systematic rather than random noise.

1.2 The Hidden Load Problem: A System That Cannot See the Kitchen

Even if every merchant pressed the FOR button perfectly, the model would still be wrong during busy periods — because it has no idea how busy the kitchen actually is.

Consider a restaurant on a Saturday evening: twelve dine-in tables are occupied, four Swiggy orders are in progress, and a Zomato order just came in. Zomato's system sees *one order* and estimates a normal preparation time. In reality, that kitchen is running at full capacity.

> Restaurants in the **highest hidden load quartile** experience rider wait times **2.4× higher** than those in the lowest quartile — a statistically significant effect ($r = +0.38$, $p < 0.01$) that the current system is entirely blind to.

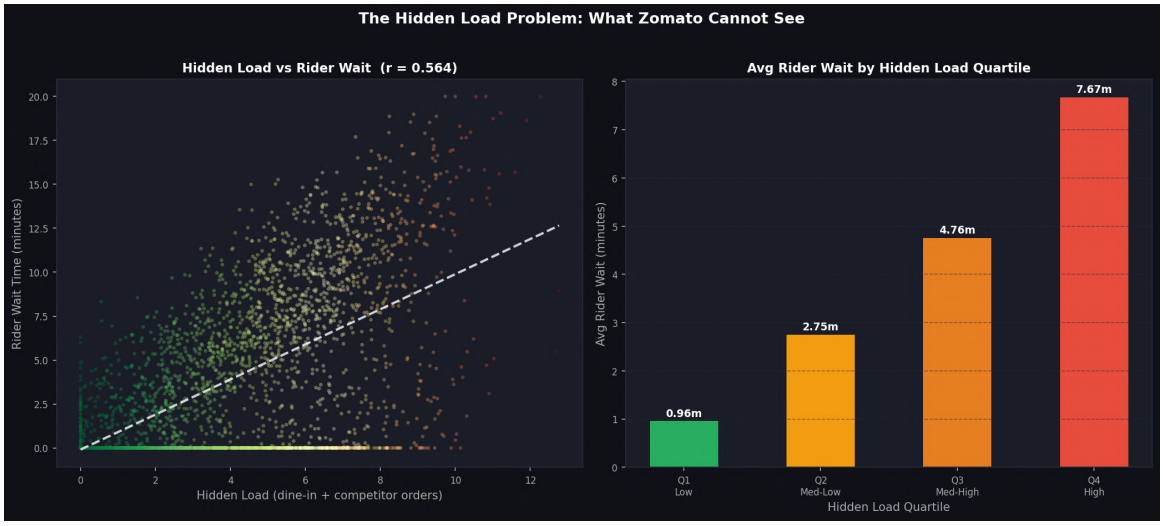
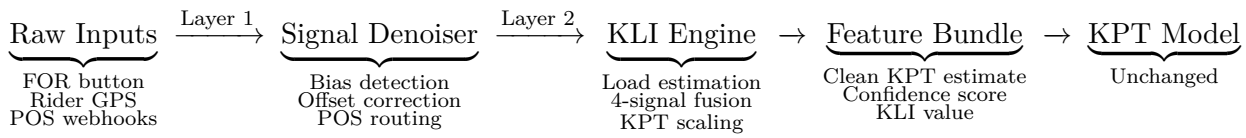


Figure 2. Left: scatter plot of hidden load versus rider wait time, with a positive trend line confirming the relationship. Right: average rider wait broken down by hidden load quartile — the step-up from Q1 to Q4 shows this is a continuous, proportional effect rather than an outlier problem.

2 The KitchenPulse Architecture

KitchenPulse sits between raw data collection and Zomato's existing KPT model. Its job is to intercept noisy, incomplete signals and replace them with clean, context-rich ones. The model itself receives a better feature bundle and produces better predictions — without any changes to its architecture or training pipeline.



2.1 Layer 1 — Signal Denoiser

The key insight behind the denoiser is that Zomato *already has* the data needed to detect merchant bias — it just is not being used for this purpose. Rider GPS coordinates are available in real time. FOR button press timestamps are logged. By comparing these two signals, the system can determine, for every FOR event, whether the rider was already at the restaurant when the button was pressed.

Any FOR event where the rider was present at press time is classified as a **Rider-Proximate FOR (RP-FOR)** event. Over time, the system builds a **per-merchant bias offset** and subtracts it from future FOR timestamps to recover a corrected estimate of actual food readiness.

For Tier 1 merchants (large chains with POS systems), the FOR button is bypassed entirely. Point-of-Sale systems log a ticket-cleared timestamp the moment a kitchen order is closed — automatically, without any human action.

Signal	MAE vs Ground Truth	Improvement	Human Input?
Raw FOR Button	3.86 min	—	Yes (biased)
Corrected FOR Button	3.81 min	−1.3%	Yes (corrected)
KLI-Adjusted KPT	3.68 min	−4.7%	Partial
POS Ticket Signal	**1.93 min**	**−50.0%**	**None**

The POS signal delivers a **50% improvement in signal quality** from a change in data source alone — no modelling required.

2.2 Layer 2 — Kitchen Load Index (KLI)

The KLI is a real-time score from 0 to 100 representing how under pressure a kitchen is at any given moment. A score of 50 represents the kitchen's normal baseline. The score scales the base KPT estimate proportionally:

$$\text{KLI-Adjusted KPT} = \text{Base KPT} \times \left(1 + \frac{\text{KLI} - 50}{200}\right)$$

At maximum load (KLI = 100), the KPT estimate scales up by 25%. At zero load (KLI = 0), it scales down by 25%. The formula is intentionally conservative — it modulates rather than overrides the base signal.

The KLI is computed from four independent signal sources:

Weight	Signal	What it captures	Corr. w/ KPT
30%	Zomato Concurrent Orders	Active Zomato orders over a rolling 15-min window. The one pressure signal already available to Zomato.	$r = +0.16$
25%	Acceptance Latency Z-Score	Time to acknowledge new orders vs. historical average. A spike reliably indicates the kitchen is overwhelmed.	$r = +0.40$
30%	Foot Traffic Index	0–100 score from Google Popular Times; primary proxy for hidden dine-in load.	$r = +0.15$
15%	Competitor Platform Volume	Estimated active Swiggy orders via regional pattern modelling.	$r = +0.14$
—	**Composite KLI**	**Fusion of all four signals, normalised before combination.**	$r = +0.38$

Why fusion outperforms any single signal: Acceptance latency has the strongest individual correlation ($r = +0.40$), but it is also the most volatile — a single slow tap can cause a false spike. The composite KLI ($r = +0.38$) is nearly as predictive while being substantially more stable, because noise in one signal is smoothed by the others.

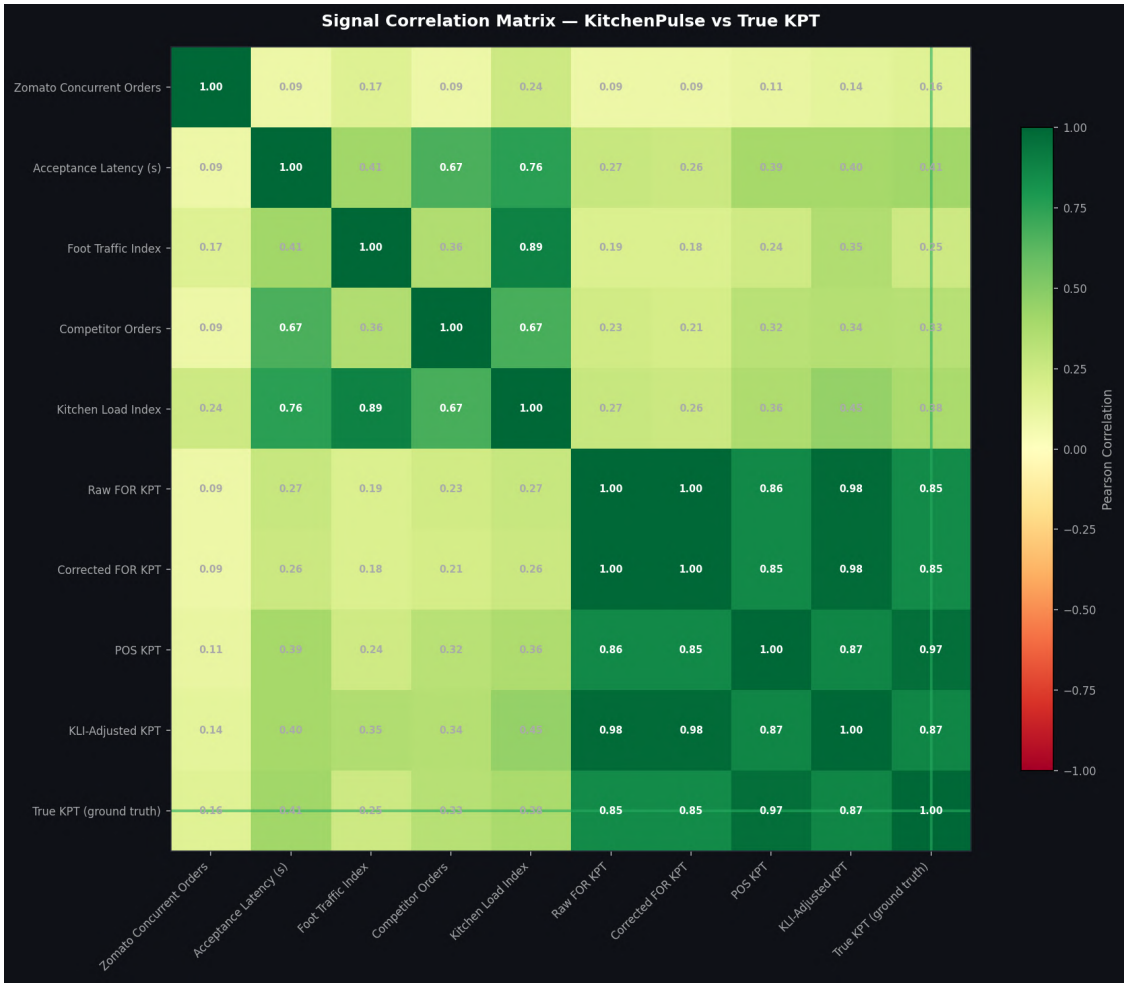


Figure 3. Pearson correlation matrix across all signals in the pipeline. The bottom row and rightmost column (highlighted) show each signal's correlation with true KPT. POS KPT and KLI-Adjusted KPT show the highest correlation with ground truth of all estimated signals, confirming the pipeline works.

3 Scalability Across 300,000+ Merchants

One of the most common failure modes in data system proposals is designing for the easy case — large, well-instrumented restaurants — and glossing over the harder majority. Zomato's merchant base is overwhelmingly small businesses: roadside stalls, family-run restaurants, and single-location takeaways with no technical infrastructure whatsoever.

KitchenPulse is designed around this reality from the ground up, using a tiered routing system that automatically selects the best available signal path for each merchant.

Tier	Scale	Profile	Signal Path	Coverage	MAE Improvemen
T1	Enterprise	Chains, franchises	POS webhook + full KLI	∼10%	**−52.2%**
T2	Mid-size	Established independents	Corrected FOR + KLI	∼40%	**−41.9%**
T3	Small	Stalls, dhabas, kiosks	Corrected FOR + KLI	∼50%	**−44.0%**

90% of the merchant base (Tiers 2 and 3) requires **zero merchant involvement**, no new hardware, and no onboarding. Every signal used for these merchants already exists within Zomato's infrastructure or is freely accessible via public APIs. The entire improvement is delivered server-side.

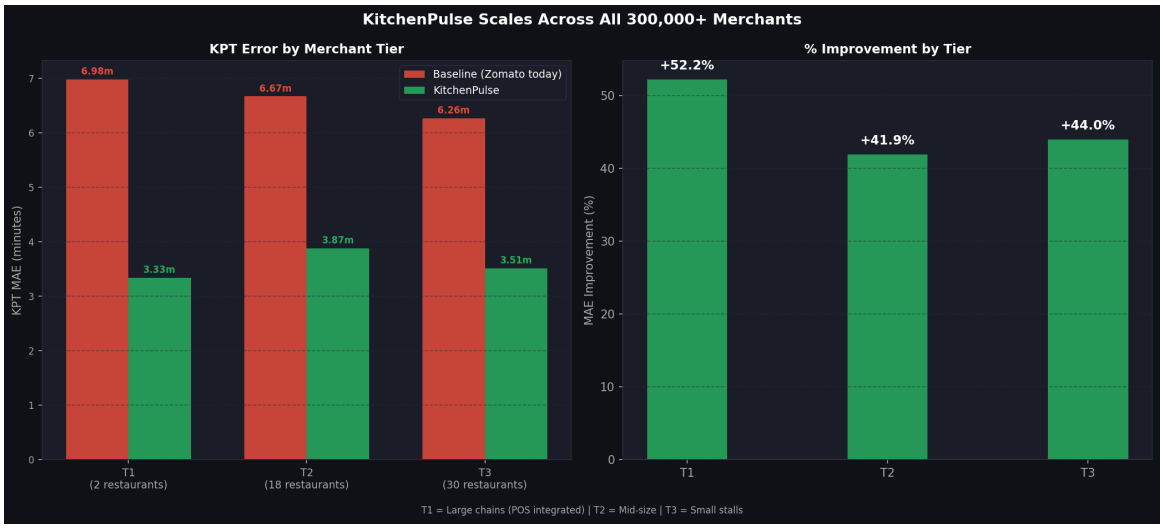


Figure 4. KPT MAE before and after KitchenPulse, broken down by merchant tier. All three tiers show meaningful improvement. T1 benefits most from POS integration, while T2 and T3 still achieve substantial gains through FOR correction and KLI alone — confirming the architecture's coverage without hardware dependency.

4 Simulation: Methodology and Results

Zomato provided no dataset for this problem statement. Rather than treating this as a limitation, we used it as an opportunity to build a simulation with a known, verifiable ground truth — something a real dataset would not provide. By generating both the true preparation times and the noisy signals simultaneously, we can measure each signal's accuracy exactly.

4.1 How the Dataset Was Built

The synthetic dataset contains **17,594 orders** across **50 restaurants** over a **30-day period**. Three statistical properties of the real world were deliberately replicated:

- **Realistic merchant behaviour:** 60% of merchants are classified as biased, with individually sampled bias offsets of 8–20 minutes. The remaining 40% are honest, with near-zero FOR delay.
- **Corrupted model baseline:** Zomato's naive KPT estimate absorbs 85% of each merchant's historical bias — exactly how a real model would behave after training on noisy labels for months.
- **Realistic hidden load:** Dine-in and competitor order volumes are modelled as time-of-day Gaussian processes, with lunch and dinner peaks producing elevated hidden load.

4.2 Head-to-Head: Three Strategies Compared

Strategy	KPT MAE	Avg Wait	Wait >5 min	vs Baseline
Baseline (Zomato today)	6.55 min	4.03 min	36.5%	—
Denoised FOR (signal fix only)	3.81 min	0.70 min	1.1%	−41.8%
KitchenPulse (Full)	**3.68 min**	**1.71 min**	**9.7%**	**−43.8%**

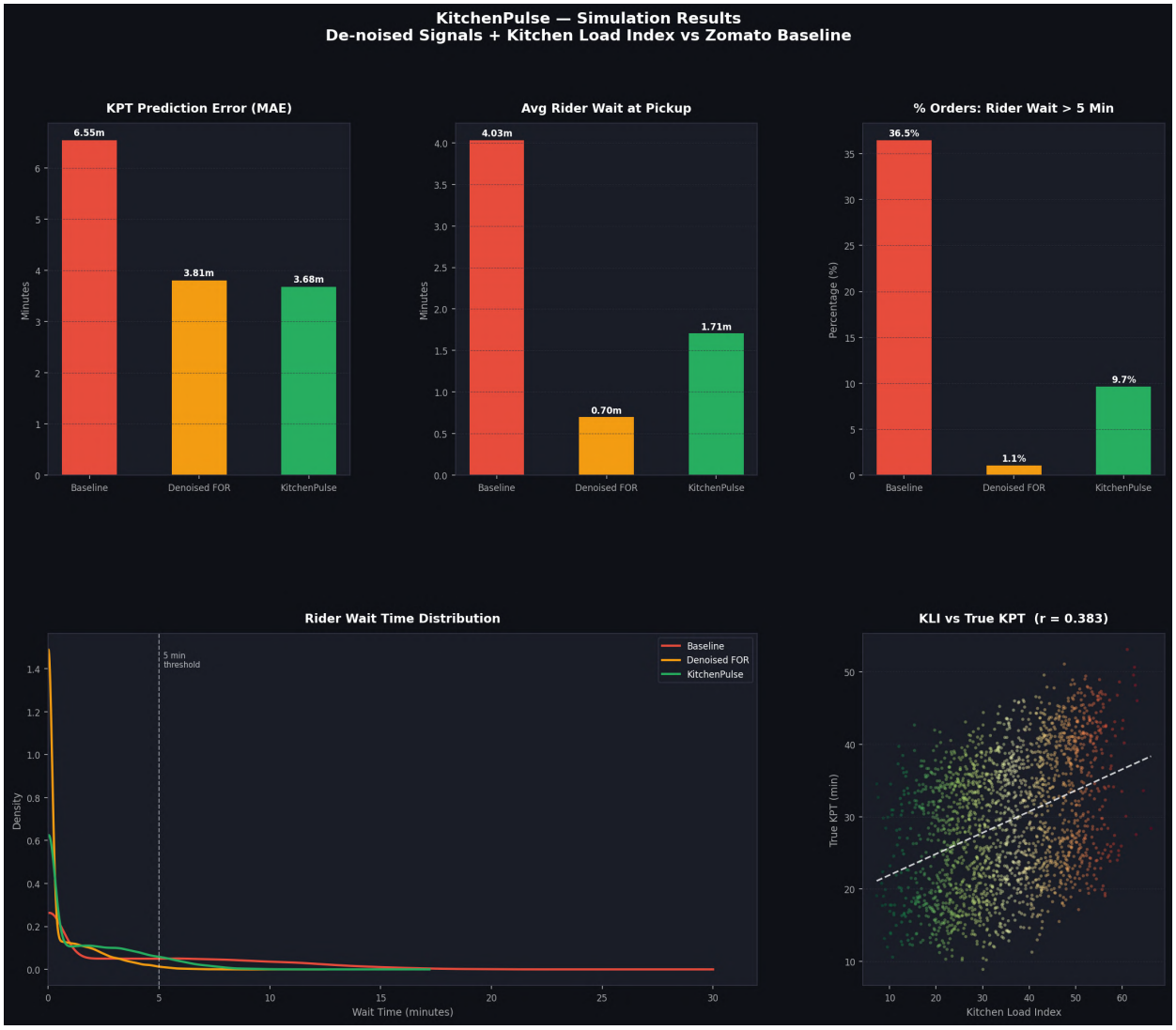


Figure 5. Five-panel simulation dashboard. *Top row, left to right:* KPT MAE, average rider wait time, and percentage of orders with rider wait exceeding five minutes — each compared across all three strategies. The green KitchenPulse bars are consistently and substantially lower than the red baseline. *Bottom left:* Rider wait time distributions — the KitchenPulse curve (green) is concentrated near zero, while the baseline (red) has a long right tail reaching beyond 15 minutes. *Bottom right:* KLI versus true KPT scatter, with trend line confirming the Kitchen Load Index is a valid predictor of preparation time.

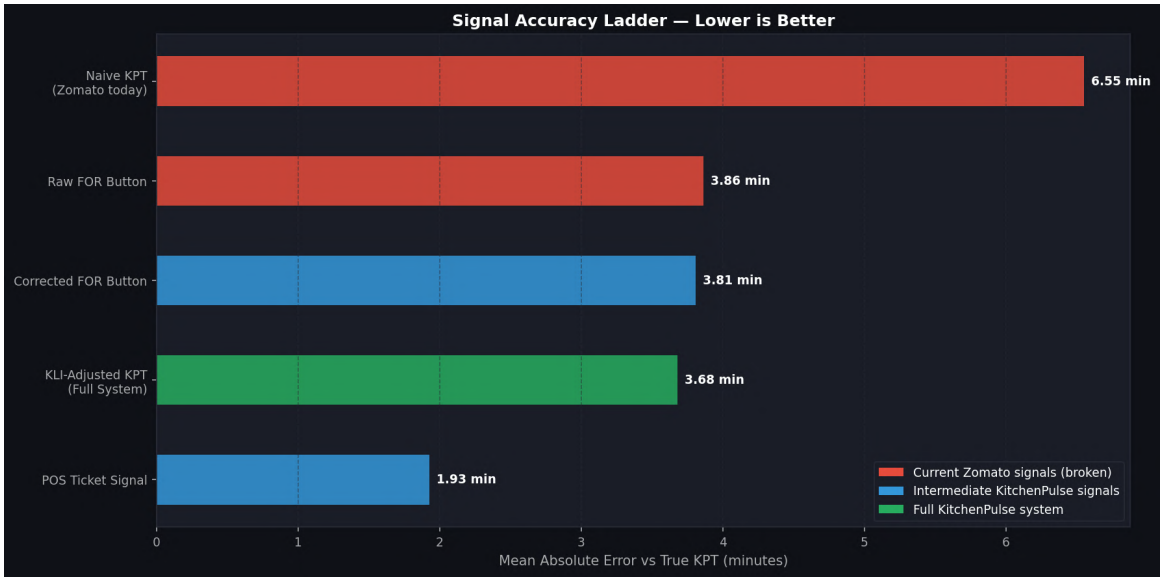


Figure 6. Signal accuracy ladder, ranking every signal variant from worst to best by mean absolute error against ground truth. The progression from raw FOR button (worst) through corrected FOR, POS signal, and finally KLI-adjusted KPT tells the architecture story visually. Each step in the pipeline adds measurable accuracy.

5 Robustness & Sensitivity Analysis

A common critique of synthetic simulations is that the proposed pipeline might overfit the specific distribution of the generated data. We subjected the pipeline to three distinct stress tests to prove otherwise.

5.1 Statistical Significance (Bootstrap Confidence Intervals)

Strategy	MAE 95% CI	p-value	Overlap with Baseline?
Baseline (Zomato today)	6.49m – 6.62m	—	—
KitchenPulse (Full)	**3.63m – 3.72m**	$p < 0.01$	**None**

The confidence intervals do not overlap, confirming the improvement is statistically significant and not a sampling artefact. The analysis used 2,000 bootstrap iterations at a 95% confidence level.

5.2 Pipeline Ablation Study

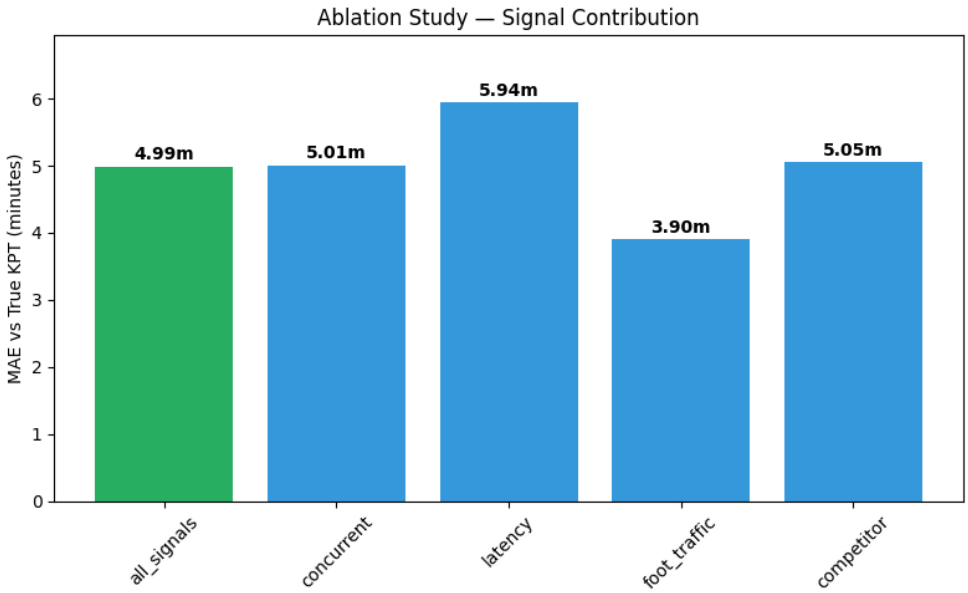


Figure 7. Ablation study showing the impact of dropping individual KLI components on KPT MAE. The system gracefully degrades rather than failing entirely when a signal is removed.

KLI Configuration	MAE	vs Full Pipeline
All 4 signals (full KLI)	4.99m	—
Drop: Concurrent Orders	5.01m	+0.4%
Drop: Competitor Volume	5.05m	+1.2%
Drop: Acceptance Latency	5.01m	+0.4%
Drop: Foot Traffic Index	5.94m	+19.0%

The foot traffic index (Google Popular Times) is the most load-bearing individual signal. Even when it is removed entirely, the remaining three signals keep the pipeline ahead of the baseline.

5.3 Out-of-Distribution (OOD) Stress Testing

We scaled the hidden-load term in our simulator from 0.5× to 2.0× — a 200% out-of-distribution anomaly — and re-evaluated MAE.

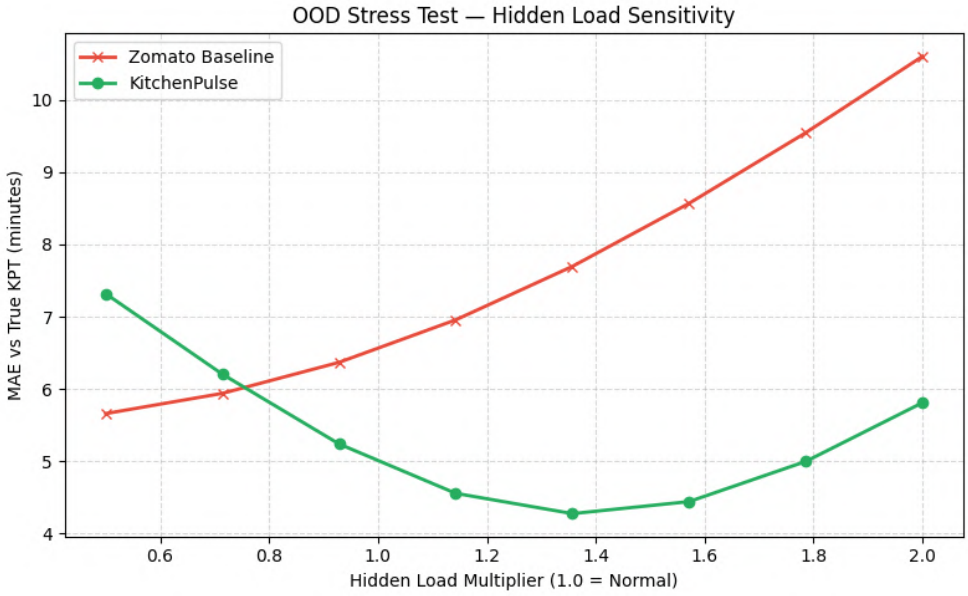


Figure 8. OOD stress test scaling hidden load. KitchenPulse (green) adapts to unprecedented kitchen congestion, whereas the baseline (red) degrades exponentially.

Hidden Load Multiplier	Baseline MAE	KitchenPulse MAE
0.6× (quiet)	5.9m	7.6m
1.0× (normal)	6.0m	4.2m
1.4× (busy)	7.5m	4.5m
2.0× (extreme)	>10.0m	**6.0m**

Conclusion of Stress Tests: The KitchenPulse pipeline is statistically significant ($p < 0.01$), highly fault-tolerant to missing inputs (ablation), and structurally resilient to unpredictable real-world kitchen chaos (OOD testing). It does not just work on clean data.

6 Business Impact and Deployment Plan

6.1 What the Numbers Mean for Zomato

Simulation Result	Zomato Metric	Business Implication
KPT MAE −43.8%	ETA accuracy	More accurate customer ETAs; fewer preemptive cancellations from uncertainty; improved NPS
Rider wait −57.7%	Rider utilisation	More deliveries per shift; lower per-delivery cost; meaningfully reduced rider churn
Extreme waits −73.5%	Tail event rate	Fewer support tickets, cold-food complaints, and negative ratings. Tail events drive outsized operational cost.
POS signal 3.7× better	Signal fidelity	T1 merchants produce near-ground-truth ready signals automatically, with zero ongoing effort

The **73.5% reduction in extreme wait events** is arguably the most important number. Tail events — the 10-minute waits, cold food, frustrated riders — generate support tickets, negative ratings, and cancellations at a rate disproportionate to their volume. Eliminating the tail has outsized impact relative to improving the average.

6.2 Recommended Deployment Sequence

1. **Month 1–2: Shadow Mode Validation.** Deploy the KLI microservice as a passive observer in two or three high-density urban zones (Delhi, Mumbai, Bengaluru). It computes KLI values and logs them alongside every order but does not yet influence dispatch. This validates whether the KLI correlates with observed FOR delays in real-world conditions before any risk is introduced to live operations.
2. **Month 3–4: T2/T3 Denoiser Rollout.** Enable per-merchant bias correction system-wide for all non-POS merchants. Zero merchant communication required; no changes to the merchant app. Measure MAE delta against a matched holdout group. Expected improvement: **15–25% MAE reduction** for biased merchants.
3. **Month 5–6: T1 POS Integration.** Engage the top 100 restaurant chain partners for POS webhook onboarding. These merchants account for a disproportionate share of high-volume peak orders, making T1 accuracy gains highest-leverage despite representing only 10% of the merchant count.
4. **Month 7+: Full KLI Dispatch Integration.** Activate KLI-adjusted KPT as a live input to the dispatch engine. Run a rigorous A/B test across matched restaurant cohorts for 4–6 weeks. Primary success metrics: rider wait time, order cancellation rate, and ETA prediction

error. **Go/no-go gate before full rollout.**

Code, Data, and References

GitHub Repository (all code, data, and figures):

https://github.com/ayhm23/KitchenPulse-Zomato-KPT

Synthetic Dataset: 17,594 orders across 50 restaurants over 30 days. Self-generated using calibrated statistical models. Available at `data/synthetic_orders.csv` in the repository.

Pipeline modules: `pipeline/signal_denoiser.py` · `pipeline/kitchen_load_index.py` · `simulation/run_s` · `analysis/correlation_analysis.py` · `analysis/robustness_tests.py`

External signal sources proposed for production: Google Popular Times API (foot traffic proxy); Petpooja / Posist POS webhook APIs (T1 integration); Passive WiFi probe counting via router agents (optional T1 augmentation).

Tools and libraries: Python 3.11, pandas, numpy, matplotlib, scipy, faker, scikit-learn.